Maastricht University

Department of Advanced Computing Science

Algorithmic Design

Course Recap (part 1)

Phew ... we covered a lot.

David Mestel and **Steven Chaplick**

2025-2026 Period 5

Topics: in the first half

Standard Algorithmic Paradigms:

- Greedy, Lectures 1 & 2:
  Grow **one** solution step-by-step according to a **clever rule**
- Divide and Conquer, Lectures 3 & 4:
  Cleverly break into pieces, solve the pieces, **combine cleverly**.
- Dynamic Programming, Lectures 5 & 6:
  Optimal solution could arise **several different ways**, try them all (recursively).

Lectures 1 and 2: Greedy Algorithms

- General References [GT §10, CLRS §16]

Problems on Intervals:

- Interval Scheduling aka Task Scheduling [GT 10.2]
- Interval Selection aka Activity Selection [CLRS 16.1]

Graph Problems:

- Minimum Spanning Trees [GT 15] [CLRS 23], Prim's algorithm [GT 15.3] [CLRS 23.2].
- Single-source Shortest Paths [GT 14] [CLRS 24], Dijkstra [GT 14.2] [CLRS 24.3]

Greedy Algorithms [GT §10, CLRS §16]

- **Greedy rule:** makes the choice that seems best based on what we did so far
- A greedy algorithm **iteratively** builds up **one** single solution by (exhaustively) "doing the what looks the best each step"

Used to solve an **Optimization problem** with a set of **configurations** and an **objective function**.

The goal is to find a configuration that maximizes (or minimizes) the value of the objective function.

Interval Scheduling [GT §10.2]

Input: A collection of time intervals $(s_1, f_1), \ldots (s_n, f_n)$, $s_i < f_i, \forall i \in \{1, \ldots, n\}$

Goal: Return a **labelling** $f : \{1, \ldots, n\} \to \{1, \ldots, k\}$ of the intervals so that no two intervals with the same label overlap, and k is as small as possible.

Greedy Rule: build schedule by earliest start time (creating a new label as needed).

k-**Partial Solution:** A solution to the interval scheduling problem on the first k intervals ordered by start time.

Correctness: Suppose Greedy is not optimal. Then opt. must use fewer labels. Consider the moment when greedy exceeds opt's labels . . . contradiction!

Interval Selection [CLRS 16.1]

Input: A collection of time intervals $(s_1, f_1), \ldots (s_n, f_n)$, $s_i < f_i, \forall i \in \{1, \ldots, n\}$

Goal: Return a **selection** $f : \{1, \ldots, n\} \to \{\texttt{Yes}, \texttt{No}\}$ of the intervals so that no two intervals mapped to `Yes` overlap and the number set to `Yes` is maximum.

Greedy Rule: in order of earliest finish time, select the next interval (set to `Yes`) if it fits and discard it (set to `No` otherwise).

k-**Partial Solution:** A solution to the interval selection problem on the first k intervals ordered by finish time.

Correctness: via an **exchange** argument with respect to an optimal solution.
Fix a selection f produced by greedy, and consider an optimal selection $f^\star$ with as many `Yes`'s in common with f as possible.
Argue that $f = f^\star$ by contradiction. E.g., suppose they are not equal and consider the first interval where f and f^* differ.

Minimum Spanning Tree (MST) [GT §15, CLRS §23]

Input: An n-vertex graph $G = (V, E)$, and edge costs $c : E \to \mathbb{R}$.

Goal: Return a tree T that *spans* G, where $\sum_{e \in E(T)} c(e)$, is minimized.

Greedy Rule: Grow T edge-by-edge, pick min. cost edge so that T remains a tree.

k-Partial Solution: An MST T of a subgraph of G where T has k edges.

Correctness: via an **exchange** argument with respect to an optimal solution.
Fix a tree T produced by greedy, and then consider an optimal solution $T^\star$ with as many edges in common as possible.
Argue that $T = T^\star$ by contradiction. E.g., suppose they are not equal and consider the first edge added by greedy that differs from $T^\star$.

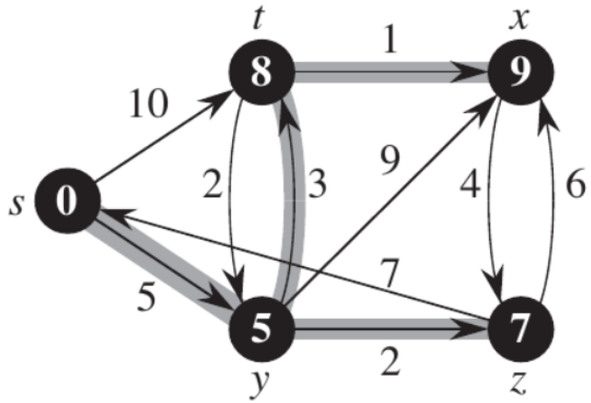
Network Routing (Shortest Paths in Graphs) [CLRS §24]

Single-Source Shortest Paths

Input: An n-vertex **directed** graph $G=(V,E)$, and positive edge weights $w{:}E\to\mathbb{R}_{>0}$.
A starting vertex $s \in V$.

Goal: Calculate a shortest path from s to every other vertex v in G.

Idea: Prefixes of shortest paths need to be shortest paths.
$\rightsquigarrow$ k-**Partial Solution**: shortest-path tree on the first k vertices.

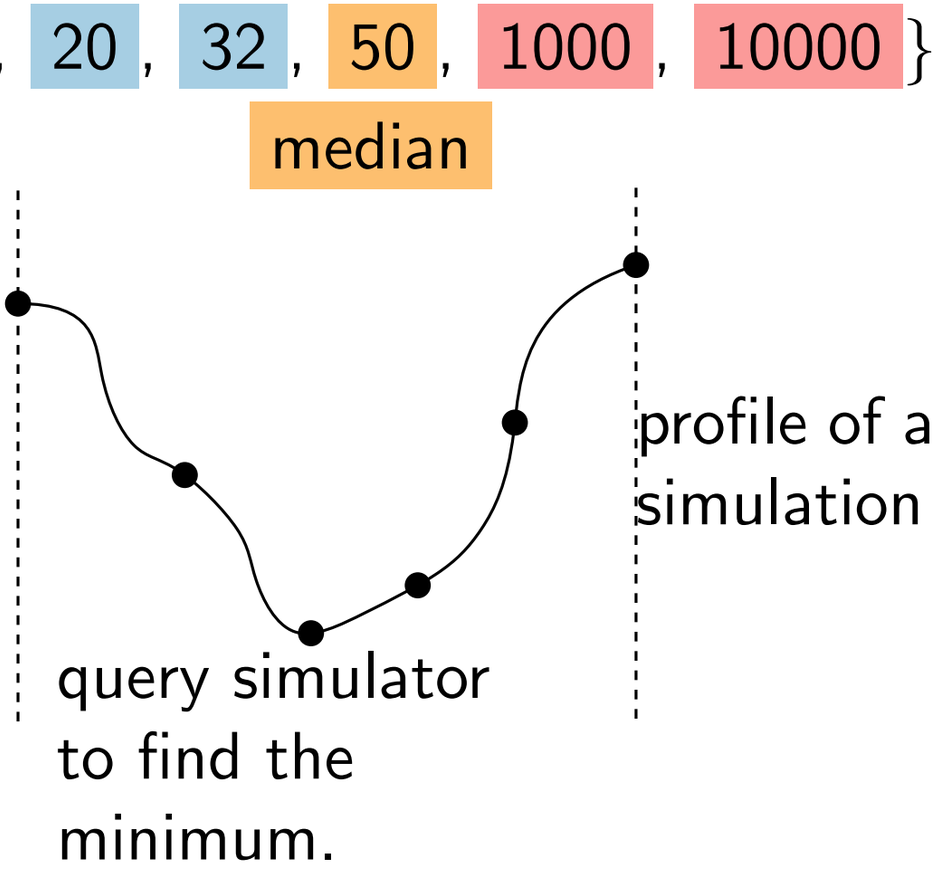


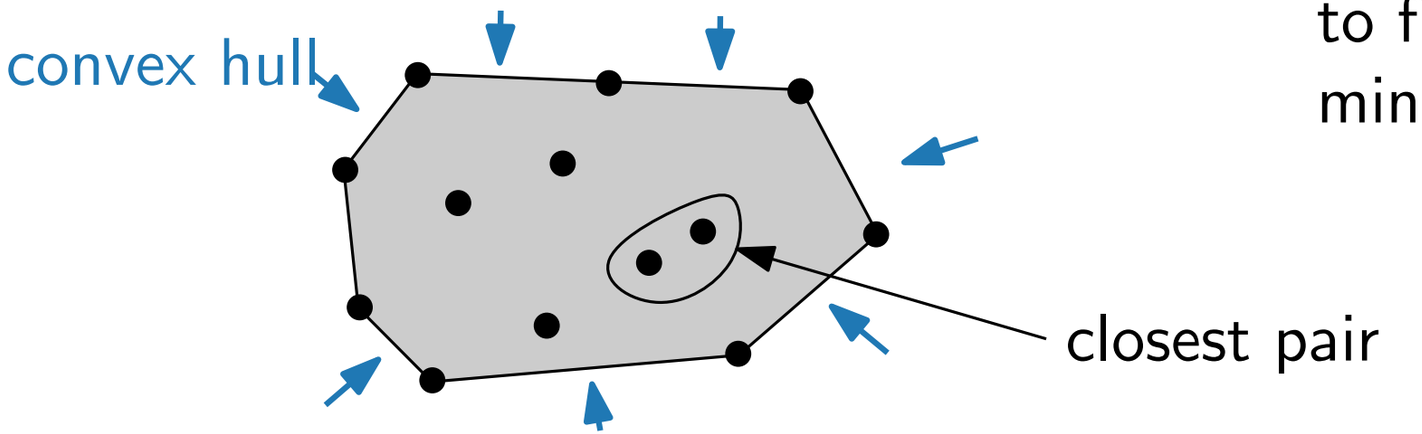
Greedy Rule: Grow shortest-path tree T by extending to the **closest** vertex to s outside of T.

Correctness: Prove by induction that T is a subtree of a shortest-path tree for the full graph G starting from s.

Lecture 4: Divide and Conquer

- Selecting the median (or k^{th} smallest) value. [GT 9.2] [CLRS 9.3]
- Minimizing a (black-box) convex function. `https://en.wikipedia.org/wiki/Golden-section_search`
- Points in $\mathbb{R}^2$. Convex hull and closest pair.





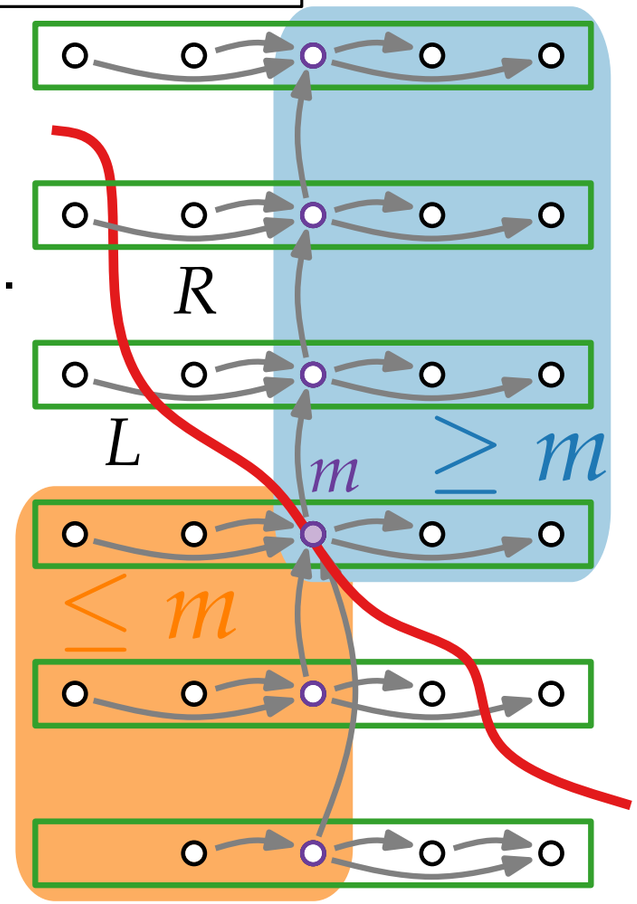
Linear Time Selection in Unordered Sets [GT 9.2] [CLRS 9.3]

Input: A *list* S of (distinct) n numbers and an index k, with $1 \leq i \leq n$.
Output: k^{th} smallest number in S, i.e., $v \in S$ with exactly $k-1$ numbers $u \in S$, $u<v$.

Theorem: For any n element set S, the runtime of $\textsc{Select}(S, k)$ is $O(n)$.

Ideas of $\textsc{Select}$.

- Split in small sets (5 elements). Find the median in each one.
- Find the median m of the medians recursively.
- Use m as a **pivot** to divide (prune) S, and recurse.

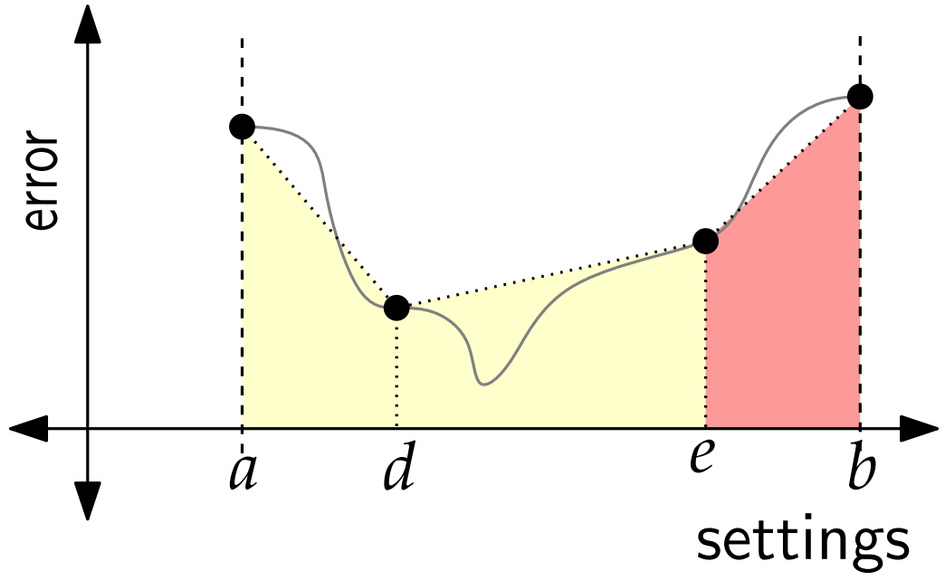


Black-box Minimization of a Convex Function:

Input: Given a convex function $f : [a, b] \to \mathbb{R}$, and a threshold $t > 0$.
(i.e., f takes in x in the real interval $[a, b]$ $(a < b)$, and returns back a real number.)
Output: $x^* \in [a, b]$ such that for any x, if $x < x^* - t$ or $x > x^* + t$, then $f(x) > f(x^*)$.

Use the 4-point idea: divide (prune)
the search the interval $[a, b]$ to find x^*.

Similar to a **binary search**, each pruning step
removes a fraction of the space.
Here at least 0.28 of the space is removed.



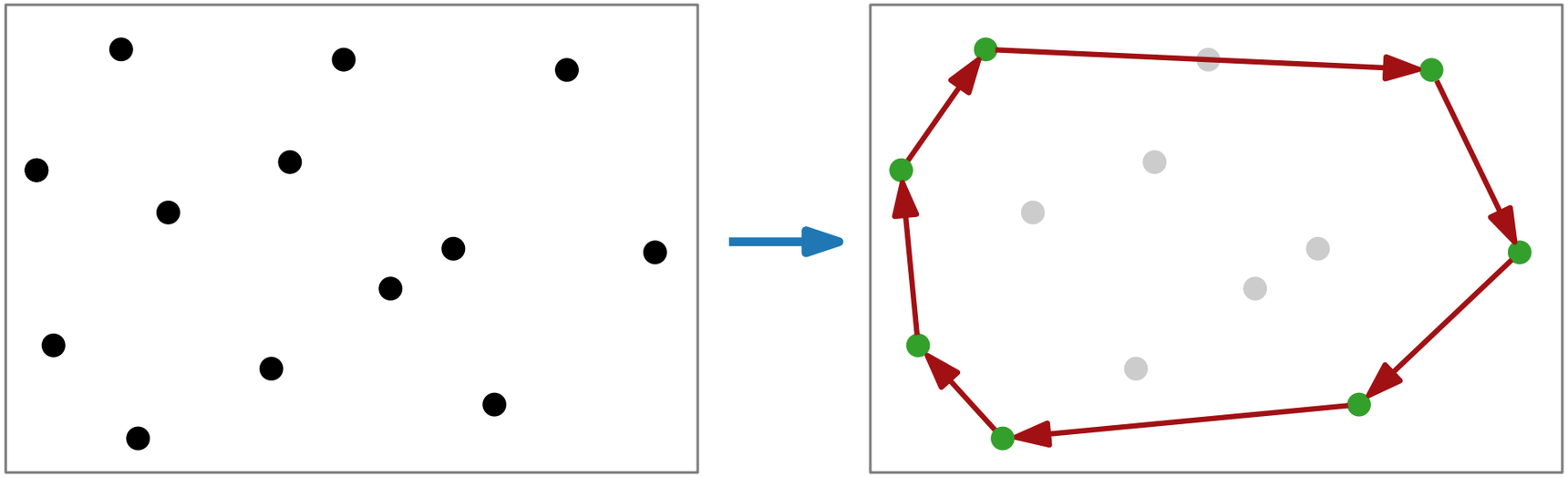
Thus, after $O(\log(\frac{a-b}{t}))$ repetitions, such an x^* can be found.

More details: `https://en.wikipedia.org/wiki/Golden-section_search`

Convex Hulls

Input: set S of n points in the plane $\mathbb{R}^2$

Output: list of corners of $\mathrm{CH}(S)$ in clockwise order



Sort points by y-coordinate. $O(n \log n)$

Recursively compute convex hull of top, and bottom half: $2T(n/2)$.

Merge by building bridges $O(n)$.

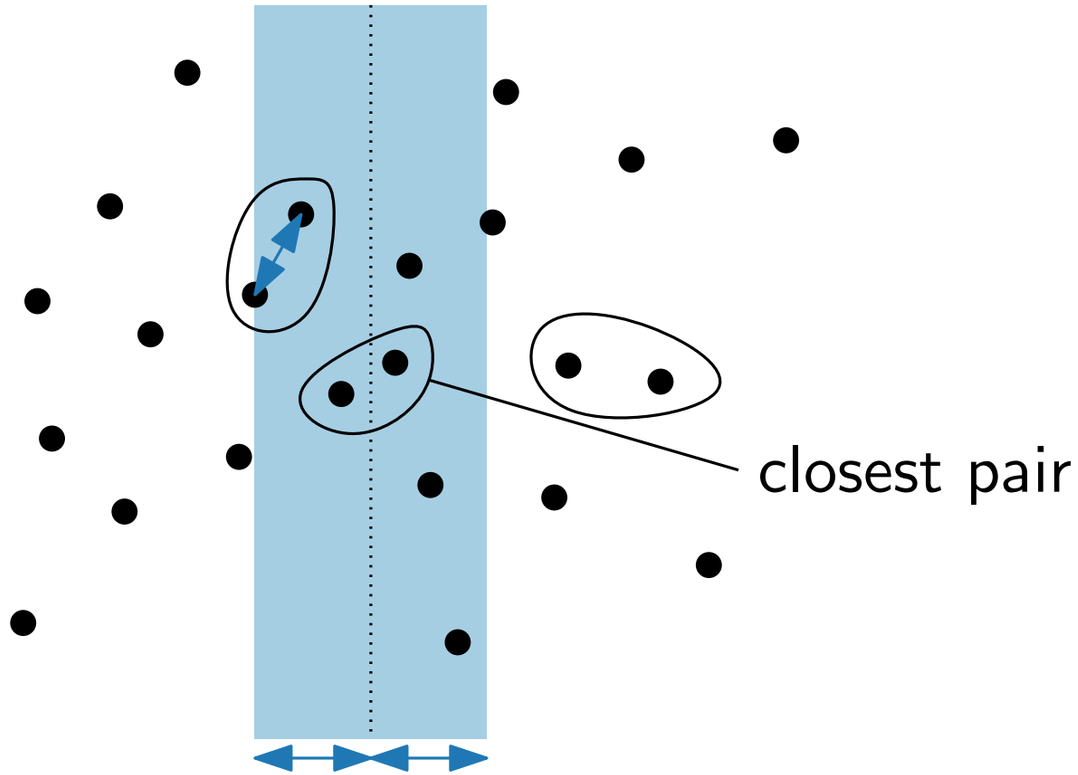
$$T(n) = 2 \cdot T(n/2) + f(n) \text{ where } f(n) \in O(n).$$

Total: $O(n \log n + T(n)) = O(n \log n)$

Closest Pair of Points

Input: A set of points in the plane, $\{p_1 = (x_1, y_1), p_2 = (x_2, y_2), ..., p_n = (x_n, y_n)\}$

Output: The closest pair of points: that is, the pair $p_i \neq p_j$ for which the distance between p_i and p_j, that is, $\sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$, is minimized.



- Split points in half, e.g., by x-coord.
- Find closest pair on left and right.
- Check the "middle" (closer than two rec. closest pairs).

Lectures 5 & 6: Dynamic Programming

General References: [GT §12], [CLRS §15]

Longest Common Subsequence (LCS) [GT 12.5 , CLRS 15.4]

- Measuring the similarity of two strings, e.g., two strands of DNA.

Scheduling Telescope Time [GT 12.3]

- Activity Selection, where activities have **benefits**

Knapsack returns [GT12.6], [CLRS 16.2 (+exercise 35-7)]

- Pseudopolynomial running time.

Travelling Salesperson (TSP)

- Finding the fastest way to deliver all packages (visit all important locations) and return home.
- Exponential running times

Roadmap for Dynamic Programming

1. **Recursive Structure:** Think of a best solution under certain conditions. For example, including or excluding a "last" element.
2. **Table:** Define an array indexed by the parameters that define subproblems, to store the optimal value for each subproblem (make sure one of the **sub**problems actually equals the whole problem).
3. **Recurrence:** Based on the recursive structure of the problem, describe a recurrence relation to compute the table values (including degenerate or base cases).
4. **Bottom-up:** Write an iterative algorithm to compute values in the array, in a bottom-up fashion, following the recurrence.
5. **Backtracking:** Use computed array values to find a solution achieving the best value (can require storing additional information about choices made while filling in the table).

Longest Common Subsequence (LCS) [GT 12.5 , CLRS 15.4]

Input: Two strings s_1, s_2. $|s_1| = n$, $|s_2| = m$.

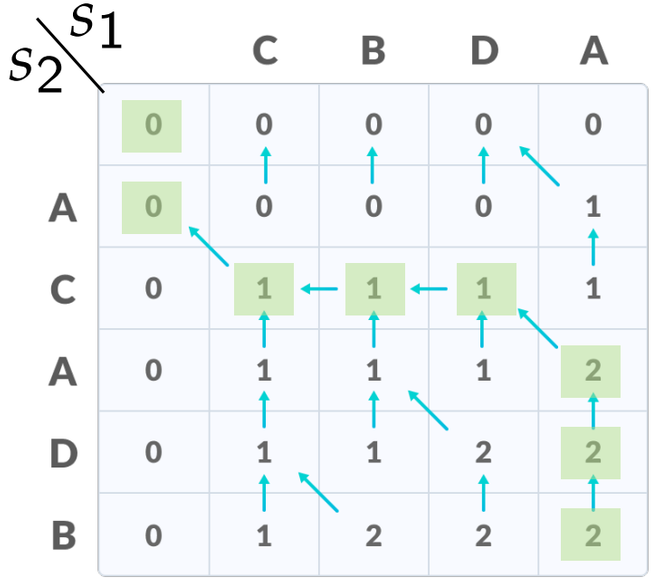
Goal: Find a string $s^\star$ of maximum length that is a subsequence of both s_1 and s_2.

Table and Subproblems: $L(i,j) :=$ LCS of prefixes $s_1[1...i]$ and $s_2[1...j]$.

Recurrance:
$$L(i,j) = \begin{cases} \max\{L(i,j-1), L(i-1,j)\} & \text{, if } s_1[i] \neq s_2[j] \\ 1 + L(i-1,j-1) & \text{, if } s_1[i] = s_2[j] \end{cases}$$

Structure: Match the last element or not.

Filling and backtracking the table. $O(nm)$ time



Scheduling Telescope Time: [GT §12.3] aka weighted interval selection

Input: A collection of time intervals $(s_1, f_1), \ldots (s_n, f_n)$, $s_i < f_i, \forall i \in \{1, \ldots, n\}$, each time interval has a positive numeric **benefit** b_i

Objective: Select a set of non-conflicting intevals to **maximize** the **benefit**.

Main result: $O(n \log n)$ time dynamic program to find an optimal solution.

- **Key idea:** Time slots can be naturally ordered by finish time.
- **Recursive Structure:** To determine if time slot i is used, look earlier in the order: best solution without i, and the best solution compatible with i.
- **Table** `OPT` indexed by time slots in this order. The value stored in the table is the optimal benefit that can be obtained by the first i time slots.

0-1 Knapsack:

Input: Given a weight capacity $W \geq 0$ and n objects with positive integer weights $w_1, \ldots, w_n$ and values $v_1, \ldots, v_n$ respectively, i.e., object i has weight w_i and value v_i.

Goal: Pick a subset S of the objects of maximum value and total weight at most W.

Main result: $O(nW)$ time dynamic program to find an optimal solution.

- **Key idea:** Indexing the table by weight limit W.
- **Recursive Structure:** To determine if object j should be used, compare dropping it (`OPT[j-1,Z]`) with keeping it and taking the best of the first `j-1` objects putting aside weight for `j` (v_j+`OPT[j-1,Z-`w_j`]`)

Remarks on runtime:

This is not polynomial time! The size of the input W is actually $\log W$

eg, $(\sum_{i=1}^{n} w_i)$, $W \in \Theta(2^n) \Rightarrow$ input size is $\Theta(n^2)$, but runtime is $O(n2^n)$

But this **kind of** the best we can do, since the problem is **NP-complete**

KNAPSACK: With repetition

Input: Given a weight capacity $W \geq 0$ and n objects with positive integer weights $w_1, \ldots, w_n$ and values $v_1, \ldots, v_n$ respectively, i.e., object i has weight w_i and value v_i.
Goal: Pick a multiset S (with repetition allowed) of the given objects of maximum value and total weight at most W.

Table: `OPT[Z]` := is the maximum value of a multiset S picked from all objects where the weight of S is at most `Z`

Recurrence:

$$\texttt{OPT[Z]} := \begin{cases} 0 & \text{if } \texttt{Z} < w_j \text{ for every } j \in \{1, \ldots, n\} \text{ (base case)} \\ \texttt{max(\{}v_j\texttt{+OPT[Z-}w_j\texttt{]}\texttt{ : } j \in \{1, \ldots, n\} \texttt{ \}) } & \text{otherwise} \\ & \text{and } w_j \leq \texttt{Z} \end{cases}$$

Table Size: $O(W)$. Computing the next entry: $O(n)$

Runtime: $O(nW)$.

Travelling Salesperson:

Input: Complete directed n-vertex graph $G = (V, E)$ with edge weights $c\colon E \to \mathbb{Q}_{\geq 0}$

Goal: An n-vertex cycle $C = (v_1, \ldots, v_n, v_{n+1}{=}v_1)$ in G, minimizing $\sum_{i=1}^{n} c(v_i, v_{i+1})$

Table and Subproblems: $\mathrm{OPT}[S, v]$ = length of the shortest s-v-path that visits precisely the vertices of $S \cup \{s\}$.

Recurrance: $\mathrm{OPT}[S, v] = \min\{\ \mathrm{OPT}[S - v, u] + c(u, v)\ \mid u \in S - v\ \}$

$$\mathrm{OPT} = \min\{\ \mathrm{OPT}[V - s, v]\ +\ c(v, s)\ \mid v \in V - s\ \}$$

Filling and Backtracking the table: $O(n^2 \cdot 2^n)$